

PTK - ZAP Lifecycle Contract and Stability Analysis

Communication contracts, automation instability root causes, and stabilization plan

Prepared from the uploaded ZAP_PTK source archive and the current PTK/ZAP lifecycle discussion.

Date: 28 May 2026

Executive Summary

The current PTK/ZAP add-on mostly follows the ZAP Pluggable Browser Extension Integration model: the bundled PTK extension talks back to ZAP through the ZAP client callback URL, and the Java add-on receives `/ptk/config`, `/ptk/progress`, and `/ptk/alert` requests through a `ClientCallBackImplementor` named `ptk`.

The unstable Chrome/Edge automation behavior is most likely caused by lifecycle-boundary issues rather than a single rule failure. ZAP Client Spider has a browser-task lifecycle; PTK has an engine/session lifecycle; findings are imported through an asynchronous publisher. A clean integration needs a contract that keeps those lifecycles separate.

- P0: Edge appears not to be explicitly registered with the PTK unpacked extension, even though Edge browser arguments are configured.
- P0: `browser_tab_safe_to_close` currently clears global zapid tracking, contradicting the intended local-tab-close contract.
- P0/P1: `postTerminalProgressForClose` checks `transport.isReady()`, but `zapTransport` exposes `isActive()`, so terminal fallback progress can fail.
- P1: Close decisions rely on WebDriver page-context JavaScript, which is fragile in Chrome/Edge/MV3/headless and during tab unload.
- P1: `/ptk/alert` acknowledges transport success but does not give enough import-by-finding reasons, so browser-side findings can be marked published even if ZAP did not import them.

1. Official ZAP Integration Requirements

The target integration should use the ZAP Pluggable Browser Extension Integration pattern. The important requirements are:

- Bundle the browser extension under `src/main/zapHomeFiles/selenium`. For Chrome/Edge, the extension must be unpacked. For Firefox, use the XPI artifact.
- Depend on the ZAP client add-on and register a `ClientCallBackImplementor`.
- Use the callback path `https://zap/zapCallBackUrl/<secret>/<implementorName>`, with `implementorName = ptk`.
- Do not call the ZAP API directly from the browser extension. The callback URL is a secret and must not leak to target pages.
- For ZAP-to-extension control, prefer a `WebSocket/control` channel after the extension has learned the callback secret.

Primary references: [ZAP Browser Extension Integration](#); [ZAP Client Side Integration Automation](#); [ZAP Selenium add-on](#)

2. Current PTK - ZAP Communication Topology

The current design is a callback-driven pipeline. ZAP launches browser workers; the PTK extension observes the ZAP callback URL; PTK posts config, progress, and findings back to the Java add-on; the Java add-on maps finding batches into ZAP alerts.

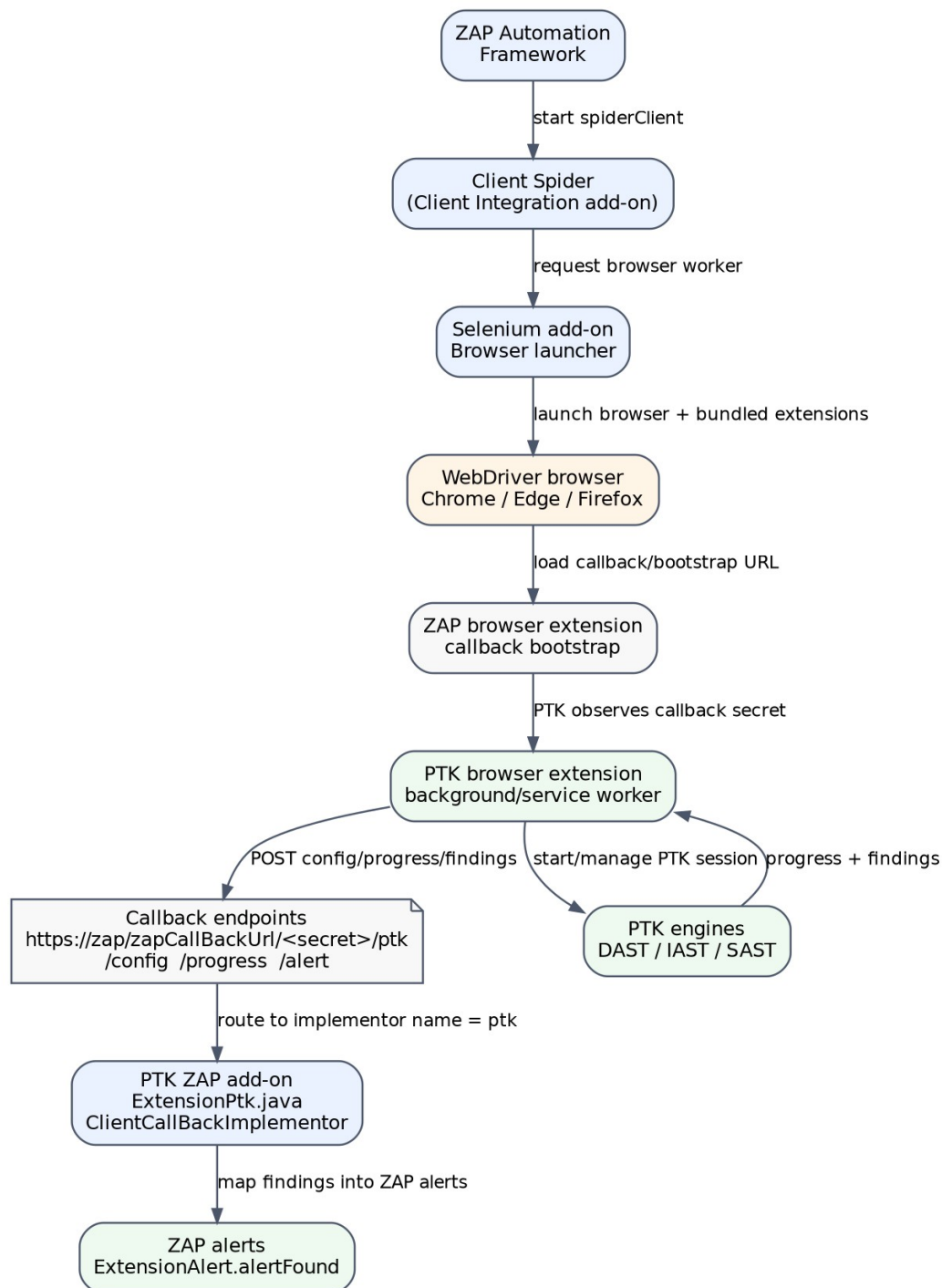


Figure 1. Current PTK - ZAP communication topology.

3. Runtime Lifecycle State Machine

The desired runtime contract should explicitly separate browser task lifecycle from PTK engine lifecycle. Callback detection is not a scan start. A browser close request is not proof that the PTK session is terminal. Terminal status should only be accepted once the publisher has drained or a bounded failure path is recorded.



Figure 2. PTK session and browser-task state model.

4. Endpoint Contracts

Endpoint	Direction	Current purpose	Stable contract needed
/ptk/config	PTK extension -> ZAP add-on	Fetch PTK mode, rulepacks, and optional ZAP history seed URLs.	Return contractVersion, zapid, browserid, configVersion, enabled engines, timeout policy, and whether config was applied.
/ptk/progress	PTK extension -> ZAP add-on	Send callback, ready, running, terminal, and safe-to-close status.	Distinguish callback_detected, config_applied, target_resolved, session_started, running,

			draining, and terminal. Include sessionId, ownerTabId, closeReadiness, and publisher state.
/ptk/alert	PTK extension -> ZAP add-on	Import PTK findings as ZAP alerts.	Include batchId, seq, sessionId, scanId. Java should return received, imported, skipped counts, and rejected reasons.
/ptk/ping	Reserved/debug	Currently debug/future-use only.	Make it a real health endpoint returning add-on version, contract version, and callback state.

Recommended progress payload shape

```
{
  "contractVersion": 2,
  "zapid": "<zap-client-callback-uuid>",
  "browserid": "<ptk-browser-instance-id>",
  "sessionId": "<ptk-session-id>",
  "ownerTabId": 123,
  "targetUrl": "http://target/",
  "phase": "running",
  "status": "running",
  "progress": 42,
  "safeToClose": false,
  "closeReadiness": {
    "scope": "global_session",
    "state": "active_work",
    "activeBrowserWork": true,
    "publisherDrained": false
  },
  "publisher": {
    "pendingFindings": 3,
    "lastBatchSeq": 12,
    "drained": false
  }
}
```

5. Source-Level Findings from the Uploaded Archive

The following findings come from the uploaded source archive. They are phrased as likely causes because the final failure depends on runtime timing, browser mode, and Client Spider scheduling.

Priority	Finding	Evidence / code area	Impact
P0	Edge extension registration appears incomplete.	ExtensionPtk.java: PTK_CHROMIUM_BROWSERS includes Browser.CHROME and Browser.EDGE, but ensurePtkSeleniumExtensionsC onfigured registers the unpacked extension for Browser.CHROME and the XPI for Browser.FIREFOX only.	Edge can receive browser arguments but may not receive the PTK unpacked extension. This can explain Edge-specific callback/config/session instability.
P0	Local tab close clears global zapid/session state.	ExtensionPtk.java: browser_tab_safe_to_close branch calls	A non-owner tab close can make later owner-session progress ignored for the same

		clearTrackingState(zapid). clearTrackingState remembers the zapid as recently closed and removes scanProgress, scanStatus, sessionId, and target URL mappings.	zapid. This can cause missing terminal progress, forced close, or missing findings.
P0/P1	Terminal close fallback readiness check is mismatched.	zapBridge.js calls this.transport.isReady?() in postTerminalProgressForClose(), while zapTransport.js exposes isActive() and no isReady().	Close fallback terminal progress can return zap_progress_not_ready and never post the terminal status Java is waiting for.
P1	Close decision is page-context dependent.	ExtensionPtk.java uses WebDriver executeAsyncScript to ask page-injected/bridge objects for PTK session progress and close readiness.	Chrome/Edge/MV3/headless runs can fail when the page is unloading, about:blank, navigating, or when the MV3 service worker/content bridge is not ready.
P1	Alert import acknowledgement is too coarse.	PtkAlertHandler returns an alert count rather than accepted/rejected finding IDs and skip reasons.	The extension can mark findings as published even if Java skipped them due to mapping, invalid URL, or missing SiteTree/history message.
P2	tabId check rejects falsy IDs.	zapBridge.js uses if (!safeTabId !targetUrl).	Usually tab IDs are positive, so this is not likely the main failure, but the contract should use safeTabId == null or safeTabId < 0.

6. Highest-Risk Failure Path

The most serious observed contradiction is that a local tab close path can clear global PTK state. This makes a multi-browser run unstable because non-owner browser tasks may complete earlier than the owner PTK session.

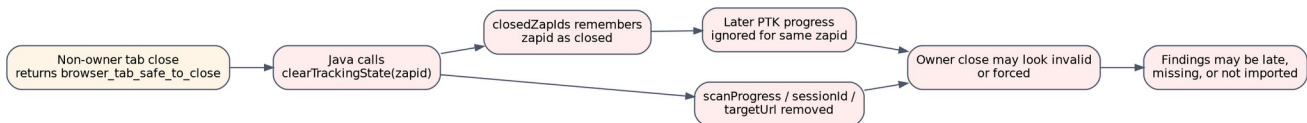


Figure 3. Current failure path caused by global state clear on local tab close.

Implementation invariant to enforce:

browser_tab_safe_to_close MUST NOT call clearTrackingState(zapid).
 browser_tab_safe_to_close MUST NOT insert zapid into closedZapIds.
 Only global safe_to_close / terminal / forced / invalid may clear global session state.

7. Desired Browser Close Contract

ZAP Client Spider can close a browser task while PTK is still doing engine-level work. The add-on must treat browserClosing as a request to close a browser worker, not as evidence that the PTK session is complete.

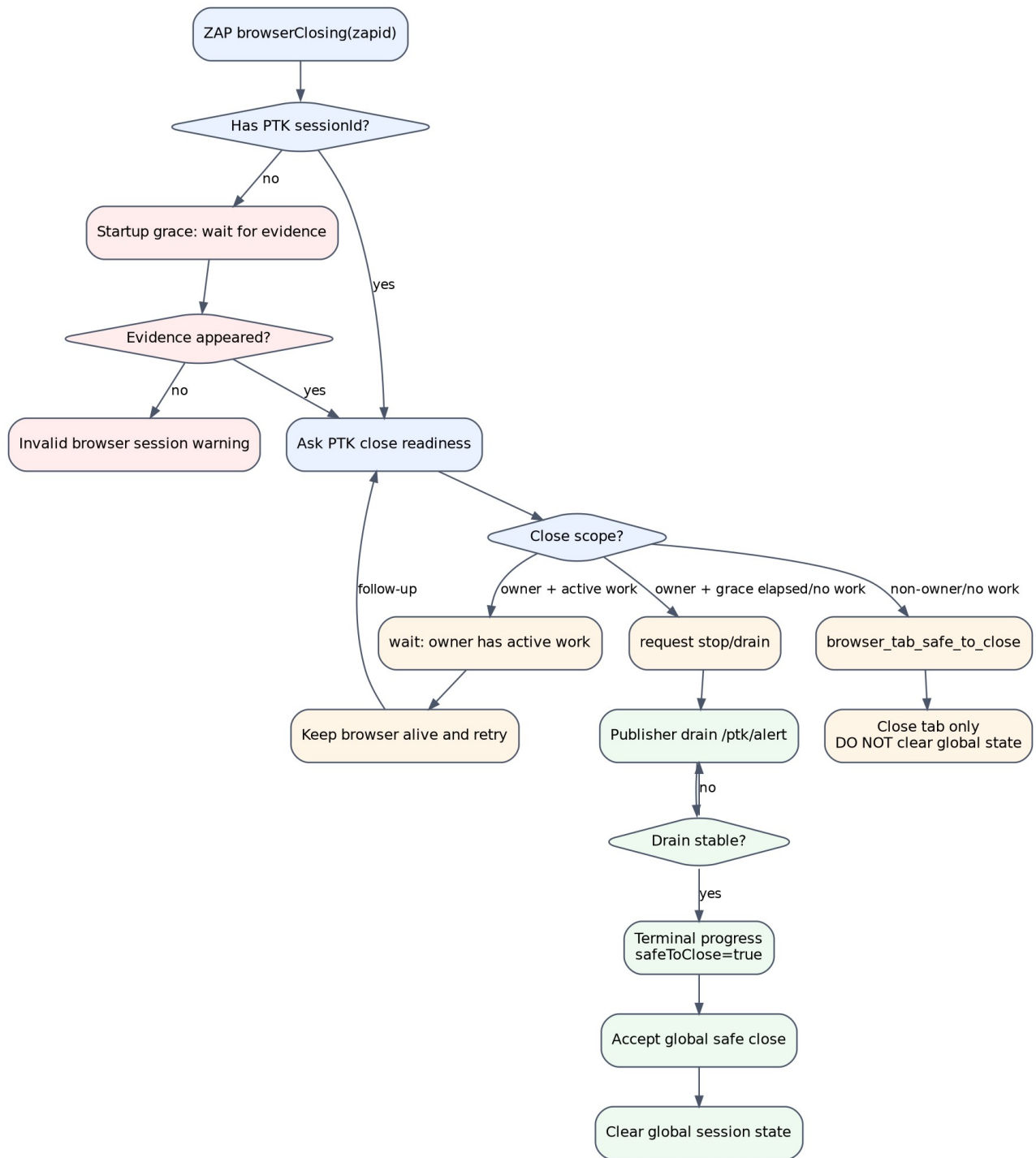


Figure 4. Desired close decision contract.

Decision	Meaning	Java behavior
browser_tab_safe_to_close	This WebDriver tab/window can close. It is local, not global.	Record local close only. Do not clear zapid/session/progress state.
wait	Owner tab still has concrete active work or terminal evidence is not available yet.	Keep browser alive within budget. Re-check readiness periodically.
safe_to_close	Global PTK session is terminal and publisher has drained, or a proven terminal close path completed.	Accept global close and clear global session state.
not_applicable / failed	Close decision could not be made.	Treat as startup/session issue unless in

		explicit manual config-only mode.
forced_closed	Close budget exhausted.	Record lifecycle warning/failure. Do not mark run as clean.

8. Better Contract Design

8.1 Split Java state into browser task state and PTK session state

The current use of a single `clearTrackingState(zapid)` makes it too easy to erase global PTK state from a local browser close path. Introduce explicit state objects and explicit clear methods.

```
class PtkZapSessionState {
    String zapid;
    String browserid;
    String sessionId;
    String targetUrl;
    Integer ownerTabId;

    boolean callbackSeen;
    boolean configServed;
    boolean sessionEstablished;
    boolean terminalProgressSeen;
    boolean publisherDrained;
    boolean globalSafeToClose;

    Set<String> closedBrowserTaskIds;
    Map<String, PtkBrowserTaskState> browserTasks;
}

clearBrowserTaskState(zapid, browserid) // local only
clearGlobalSessionState(zapid, sessionId) // terminal/global only
markInvalidBrowserTask(zapid, browserid, reason)
markForcedGlobalClose(zapid, sessionId)
```

8.2 Add a PTK wait/barrier after spiderClient

Blocking `browserClosing` helps, but it is not the only barrier needed. The automation plan or release checker should wait for PTK sessions to become terminal and for the publisher to drain after Client Spider completes.

```
- type: spiderClient
parameters:
  browserId: chrome-headless
  numberOfBrowsers: 5
  pageLoadTime: 5
  shutdownTime: 20

- type: ptk-wait
parameters:
  target: "http://target/"
  minSessions: 1
  timeout: 180
  requireTerminal: true
  requireNoForcedClose: true
  requirePublisherDrained: true
```

8.3 Prefer WebSocket for ZAP -> PTK control

The current close decision path goes through WebDriver and page-context JavaScript. A more robust design is a direct add-on-to-extension control channel opened from the PTK background/service worker after the callback secret is known.

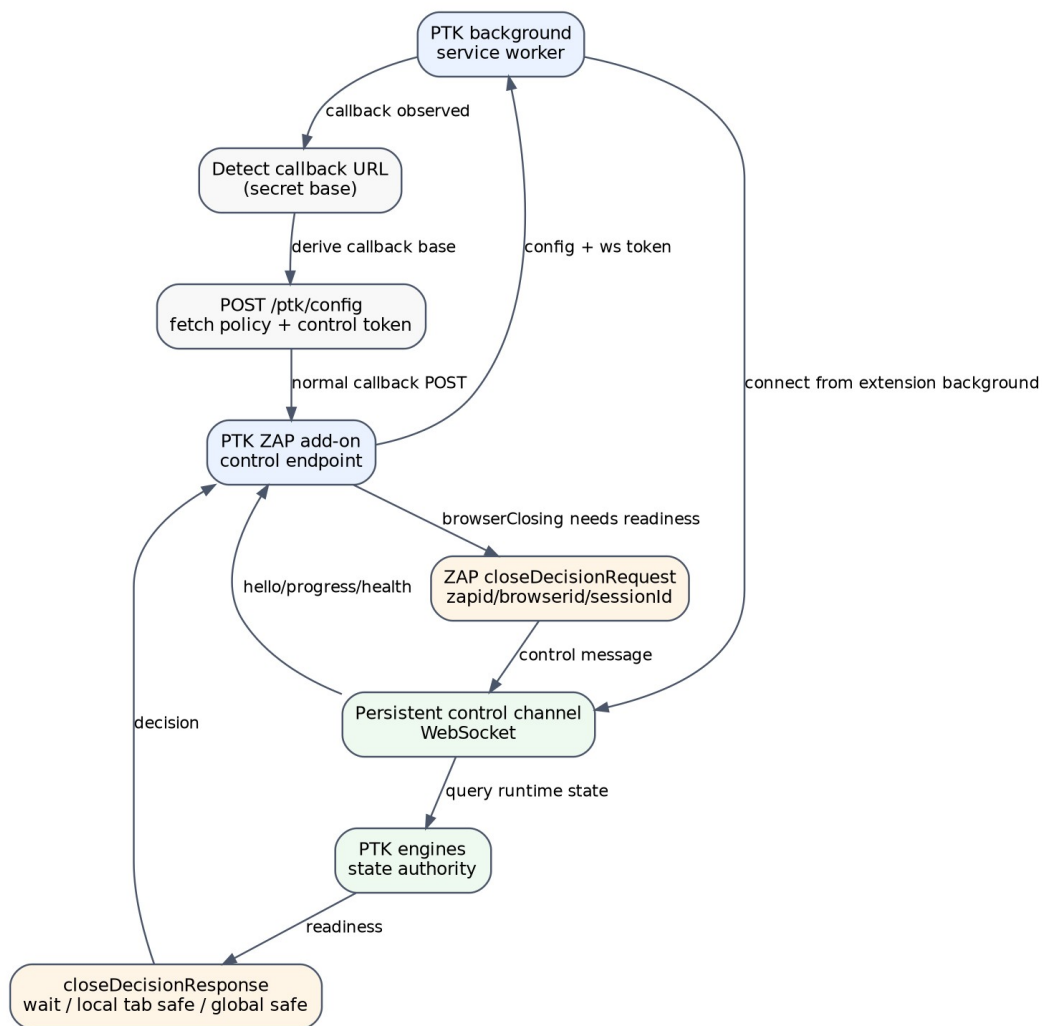


Figure 5. Recommended future WebSocket/control-channel contract.

9. Immediate Development Checklist

Order	Change	Expected effect
1	Register the unpacked PTK extension for Browser.EDGE in ensurePtkSeleniumExtensionsConfigured.	Makes Edge behave like Chrome instead of receiving only arguments.
2	Replace clearTrackingState(zapid) in the browser_tab_safe_to_close path with recordBrowserLocalClose(zapid,browserid,decision).	Prevents non-owner tab closes from invalidating the active global PTK session.
3	Add zapTransport.isReady() as an alias for isActive(), and/or change postTerminalProgressForClose() to call isActive().	Allows close fallback to post terminal progress.

4	Change the tab ID check from !safeTabId to safeTabId == null safeTabId < 0.	Tightens JS contract correctness.
5	Add structured PTK_CONTRACT logs for each lifecycle phase.	Lets the release checker pinpoint which boundary failed.
6	Return structured import results from /ptk/alert.	Differentiates engine misses from Java import/mapping/SiteTree misses.
7	Add a PTK wait/barrier after spiderClient.	Prevents declaring a run finished before PTK drains.
8	Move close readiness from WebDriver page JS to background/WebSocket control channel.	Long-term fix for Chrome/Edge/MV3/headless close races.

10. Test Plan

10.1 Unit tests

Area	Test	Pass condition
Java	Edge registration	Selenium browser extensions contain PTK entries for Chrome, Edge, and Firefox.
Java	Local tab close does not clear session	scanProgress, scanStatus, sessionIdByZapId, targetUrlByZapId remain after browser_tab_safe_to_close.
Java	Late progress after local close	A /ptk/progress update for the same zapid is accepted, not ignored as recently closed.
Java	Global safe close clears state	Only safe_to_close/terminal/forced/invalid paths clear global state.
Java	Alert import reasons	Missing mapping, missing SiteTree match, invalid URL, and successful raw-message import are each reported correctly.
JS	Terminal close readiness	postTerminalProgressForClose succeeds when transport is active.
JS	Close grace decisions	owner+active+grace => wait; non-owner+active => browser_tab_safe_to_close; terminal => safe_to_close.
JS	Config fallback	Config fetch failure is logged as degraded and not mistaken for ZAP-configured success.

10.2 Integration matrix

Browser	Mode	numberOfBrowsers	Required evidence
Chrome	headed	1	callback, config, session established, no forced close, expected import.
Chrome	headless	1	same as headed baseline.
Chrome	headless	5	at least one worker and one PTK session; lower fan-out is warning, not fail.
Edge	headed	1	PTK extension visible and callback/config/progress observed.
Edge	headless	1	same as Edge headed.
Edge	headless	5	primary regression gate; no local close clears global state.
Firefox	headless	1	comparison baseline.

Firefox	headless	5	comparison baseline for multi-worker close behavior.
---------	----------	---	--

11. Revised Release Gates

Gate	Must pass?	Reason
At least one ZAP-owned browser worker loaded the target	Yes	Proves Client Spider browser coverage started.
PTK callback detected	Yes	Proves the ZAP/extension callback bridge started.
Config served and applied	Yes unless fallback is explicitly tested	Proves ZAP-controlled PTK policy was applied.
PTK session established with non-empty sessionId	Yes	Callback progress alone is not a scan.
No forced_closed	Yes	Forced close means lifecycle uncertainty.
No global state cleared on local tab close	Yes	Prevents multi-browser truncation.
Publisher drained before terminal	Yes	Prevents missing late findings.
Expected findings imported or rejected with reason	Yes	Separates engine failure from import/mapping/SiteTree failure.

12. Structured Lifecycle Logs

Add a single structured log family across Java and JS so that automation reports can reconstruct the exact failure boundary.

```
PTK_CONTRACT zapid=... browserid=... sessionId=... phase=... status=... targetUrl=...
```

Suggested phases:

```
browser_launched
callback_detected
callback_progress_posted
config_requested
config_served
config_applied
target_resolved
session_start_requested
session_established
progress_running
alert_batch_sent
alert_batch_imported
close_requested
close_decision
publisher_drained
terminal_progress_posted
global_close_accepted
local_tab_closed
forced_closed
```

13. Corrections to the Lifecycle Contract Document

The Codex lifecycle document is directionally correct, but the implementation should be made to match its most important invariant: `browser_tab_safe_to_close` is local, not global.

- Add an explicit invariant: browser_tab_safe_to_close must not call clearTrackingState(zapid).
- State that closedZapIds is a global close guard and must not be used for local tab closes.
- Clarify that terminal progress from close fallback depends on zapTransport readiness and must call isActive/isReady consistently.
- Clarify that numberOfBrowsers is max concurrency and the release checker should fail only on no worker/session or lifecycle violations, not lower-than-max fan-out.
- Add a failure-boundary triage table: browser load -> callback -> config -> target resolve -> session -> finding generated -> alert imported -> report interpreted.

14. Recommended Fix Order

- Register Edge extension explicitly.
- Stop clearing global zapid/session state on browser_tab_safe_to_close.
- Fix the postTerminalProgressForClose readiness check.
- Fix the tabId check.
- Add structured lifecycle logs and update the weekly checker to assert phases.
- Add import-result reasons for /ptk/alert.
- Add a PTK wait/barrier after spiderClient.
- Move close readiness from WebDriver page JS to a WebSocket/background-channel contract.

Appendix A - Minimal Patch Sketches

A.1 Edge extension registration

```
changed |= ensureBrowserExtension(extensions, chromiumPath, Browser.CHROME, "Chromium");
changed |= ensureBrowserExtension(extensions, chromiumPath, Browser.EDGE, "Edge");
changed |= ensureBrowserExtension(extensions, xpiPath, Browser.FIREFOX, "Firefox");
```

A.2 Local tab close should not clear global state

```
if (PtkCloseContract.isBrowserLocalTabSafeToCloseDecision(closeDecision)) {
    logCloseContractDecision(zapid, browserid, "browser_tab_safe_to_close", closeDecision);
    logTimingSummary(zapid, browserid, "browser_close.local_tab", elapsed, closeExtra);
    recordBrowserLocalClose(zapid, browserid, closeDecision);
    return;
}

private void recordBrowserLocalClose(String zapid, String browserid, Map<String, Object> decision) {
    // Local bookkeeping/logging only.
    // Do NOT call rememberClosedZapId.
    // Do NOT remove scanProgress, scanStatus, sessionIdByZapId, targetUrlByZapId.
}
```

A.3 Transport readiness alias

```
// zapTransport.js
isReady() { return this.isActive() }

// zapBridge.js
if (!sessionId || !effectiveZapId || !this.transport.isActive?.()) {
    return { ok: false, posted: false, reason: 'zap_progress_not_ready' }
}
```